# Lecture 5b: Learning the Network Part 3 - Backpropagation (2) - Vector Formulation


1 Recap
2 Vector Calculus
3 Back Calculus for Backpropagation
4 Backpropagation


## 1 Recap

- Backpropagation is best done using vector formulation
- The neuron is any MA layer: gets a bunch of affine values $z$ given $\mathbf{y}$ to compute $\mathbf{z}$
- $\mathbf{z}$ is represented as a vector, $\mathbf{y}_{\mathbf{z}}$ represents the layer, $\mathbf{y}_{\mathbf{z}}$ represents the width of the MA layer

- $\mathbf{y}_{\mathbf{z}}$ is just through activations and we're going to get vector $\mathbf{y}_{\mathbf{z}}$
- $\mathbf{y}_{\mathbf{z}}$ are the set of $y$'s which come out of the MA layer

- $y$'s from the $L-1$ layer are used to compute the $z$ values in the $L$th layer

- Associated with these connections are the weights
- $\mathbf{W}^L_{jk}$ is the weight on the output of the $k$th neuron in the $L-1$ layer to the $j$th neuron in the $L$th layer. So the scalar value of $z$ of the $j$th neuron in the $L$th layer is

$$z^L_j = \sum_k \mathbf{W}^L_{jk} y^{L-1}_k + b^L_j$$

- So this $\mathbf{z}_L$ in vector form:

- For all $z$'s of the $L$th layer:
  - $z$'s are just the common
  - For each $z$ $\exists$ need to add a corresponding bias
  - The $\exists$ is now represents the set of incoming weights to the from neuron and so on
  - In the weight matrix row! have as many columns as the number of neurons in the previous layer and as many rows as the number of neurons in the current $L$th layer

- $\mathbf{y}_L$ is the activation function applied to $\mathbf{z}_L$
- the while function is a vector function: activation applied on affine functions. The inner most variable is the input

## 1 Vector Calculus

- Derivative of a vector is $\nabla_{\mathbf{z}}\mathbf{z}$
- Derivative of a vector function is a Jacobian
- If $y$ is a scalar or a vector - its going to be a row vector, a transpose of the gradient
- General Chain rule

$$\Delta z = \frac{\partial z}{\partial y}\Delta y$$

- We cannot change the order, right multiply the vectors going forward
- The case must match

**Vector Calculus Note 1: Definitions**

- $\mathbf{z}$ is a scalar function of a vector variable
  - $\mathbf{z}$ is a function of many variables, $\mathbf{z}$ is a function of $\mathbf{y}$ and $\mathbf{y}$ is a vector, $\mathbf{z}$ is a function of many variables
  - $\mathbf{z}$ is a scalar function of a vector argument
  - $\mathbf{z} = f(\mathbf{y}) = f(y_1, y_2, \ldots, y_n)$
  - $\mathbf{z} = \mathbf{y}_1 \mathbf{y}_2 + \mathbf{y}_3 \mathbf{y}_4 + \mathbf{y}_5 \mathbf{y}_6$
  - $\nabla_{\mathbf{y}} \mathbf{z} = \left[\frac{\partial \mathbf{z}}{\partial \mathbf{y}_1}, \frac{\partial \mathbf{z}}{\partial \mathbf{y}_2}, \ldots, \frac{\partial \mathbf{z}}{\partial \mathbf{y}_n}\right]$
  - $\nabla_{\mathbf{y}} \mathbf{z} = [\mathbf{y}_2, \mathbf{y}_1, \mathbf{y}_4, \mathbf{y}_3, \mathbf{y}_6, \mathbf{y}_5]$
  - $\nabla_{\mathbf{y}} \mathbf{z}$ is a vector of size $n$ (the size of $\mathbf{y}$)
  - $\nabla_{\mathbf{y}} \mathbf{z}$ is called the gradient of $\mathbf{z}$ w.r.t $\mathbf{y}$
  - $\nabla_{\mathbf{y}} \mathbf{z}$ is called the Jacobian of $\mathbf{z}$ w.r.t $\mathbf{y}$
- Since $\mathbf{z}$ is a vertical stacking of the components, the Jacobian $\mathbf{z}$ is a vertical stacking of derivatives of $\mathbf{z}$ and $\mathbf{y}$ vector. Derivative of individual components of $\mathbf{z}$

**Calculus Notes: The Jacobian**

- The derivative of a vector function is a vector that is called the Jacobian
  - $\mathbf{z}$ is a function of several variables and $\mathbf{y}$ is a vector
  - $\mathbf{z} = \mathbf{y}_1 \mathbf{y}_2 + \mathbf{y}_3 \mathbf{y}_4 + \mathbf{y}_5 \mathbf{y}_6$
  - Using chain rule $\frac{\partial \mathbf{z}}{\partial \mathbf{y}_1} = \mathbf{y}_2$
  - Gradient $\nabla_{\mathbf{y}} \mathbf{z} = [\mathbf{y}_2, \mathbf{y}_1, \mathbf{y}_4, \mathbf{y}_3, \mathbf{y}_6, \mathbf{y}_5]$
  - Jacobian $\frac{\partial \mathbf{z}}{\partial \mathbf{y}} = [\mathbf{y}_2, \mathbf{y}_1, \mathbf{y}_4, \mathbf{y}_3, \mathbf{y}_6, \mathbf{y}_5]$

**Vector Calculus Note 1: Definitions**

- $\mathbf{z}$ is a scalar function of a vector argument
  - $\mathbf{z} = f(\mathbf{y})$
  - $\mathbf{z} = \mathbf{y}_1 \mathbf{y}_2 + \mathbf{y}_3 \mathbf{y}_4 + \mathbf{y}_5 \mathbf{y}_6$
  - $\nabla_{\mathbf{y}} \mathbf{z} = [\mathbf{y}_2, \mathbf{y}_1, \mathbf{y}_4, \mathbf{y}_3, \mathbf{y}_6, \mathbf{y}_5]$
  - $\nabla_{\mathbf{y}} \mathbf{z}$ is a vector of size $n$ (the size of $\mathbf{y}$)
  - $\nabla_{\mathbf{y}} \mathbf{z}$ is called the gradient of $\mathbf{z}$ w.r.t $\mathbf{y}$
  - $\nabla_{\mathbf{y}} \mathbf{z}$ is called the Jacobian of $\mathbf{z}$ w.r.t $\mathbf{y}$
  - $\frac{\partial \mathbf{z}}{\partial \mathbf{y}} = [\mathbf{y}_2, \mathbf{y}_1, \mathbf{y}_4, \mathbf{y}_3, \mathbf{y}_6, \mathbf{y}_5]$

## 2 Vector Calculus for Backpropagation

- Jacobian is a vector function of a vector function. Jacobian influences the corresponding $z_j$ so the Jacobian is going to be a diagonal matrix. So vector activations is a vector such as ReLU, inflexion, etc.

**Jacobian can describe the derivative of vector activations w.r.t their input**

- Affine is a fully connected layer
- Activation is a vector function
- Jacobian is a diagonal matrix (because $y_i$ is only a function of $z_i$ and not other $z$'s)

**For Vector activations**

- Affine is a fully connected layer
- Activation is a vector function
- Jacobian is not a diagonal matrix

- Regardless of whether it's a full matrix or not, the following property must hold:

$$\Delta \mathbf{y} = J_y(z)\Delta z$$

- $J_y(z)$ is the Jacobian of $\mathbf{z}$ w.r.t $\mathbf{y}$
- We know from the equation $z = \mathbf{W}\mathbf{y} + b$ that the derivative of $\mathbf{z}$ w.r.t $\mathbf{W}$ is just $\mathbf{W}$

**Special case: Affine functions**

$$z(j) = \mathbf{W}_j \mathbf{y} + b$$

$$\nabla_{\mathbf{y}} \mathbf{z}(j) = \mathbf{W}$$

- Jacobs $\mathbf{W}$ and bias $b$ operating on vector $\mathbf{y}$ to produce vector $\mathbf{z}$
- The derivative of $\mathbf{z}$ w.r.t $\mathbf{y}$ is simply the matrix $\mathbf{W}$

**Vector Calculus Note 2: Chain rule**

- For nested functions we have the following chain rule

$$\mathbf{z}(\mathbf{y}(\mathbf{x})) \implies \nabla_{\mathbf{x}} \mathbf{z}(\mathbf{y}(\mathbf{x}))$$

| | |
|---|---|
| Step 1 | $\mathbf{y} = \mathbf{y}(\mathbf{x})$ |
| Step 2 | $\mathbf{z} = \mathbf{z}(\mathbf{y})$ |
| Step 3 | $\mathbf{z} = \mathbf{z}(\mathbf{y}(\mathbf{x}))$ |

Note bene: The derivative of the chain function is the product of the derivatives of the individual functions

**Vector Calculus Note 2: Chain rule**

- Chain rule for Jacobians
- For vector functions of vector inputs:

$$\mathbf{z}(\mathbf{y}(\mathbf{x})) \implies \nabla_{\mathbf{x}} \mathbf{z}(\mathbf{y}(\mathbf{x}))$$

| | |
|---|---|
| Step 1 | $\mathbf{y} = \mathbf{y}(\mathbf{x})$ |
| Step 2 | $\mathbf{z} = \mathbf{z}(\mathbf{y})$ |
| Step 3 | $\mathbf{z} = \mathbf{z}(\mathbf{y}(\mathbf{x}))$ |

Note bene: The derivative of the chain function is the product of the derivatives of the individual functions

- In python vector notation is used, everything is transposed, and it then uses the order of multiplication is from right to left.
- When we have a scalar function of a vector function of a vector:

**Vector Calculus Note 2: Chain rule**

- Combining Jacobians and Gradients
- For scalar functions of vector inputs (z) is scalar):

$$z = f(\mathbf{y}(\mathbf{x})) \implies \nabla_{\mathbf{x}} z = J_{\mathbf{x}}(\mathbf{y}) \nabla_{\mathbf{y}} z$$

| | |
|---|---|
| Check | $[J] = [J_{\mathbf{x}}(\mathbf{y})]$ |
| | $[\nabla] = [\nabla_{\mathbf{y}} z]$ |
| | $[J] \cdot [\nabla] = [J_{\mathbf{x}}(\mathbf{y})] \cdot [\nabla_{\mathbf{y}} z]$ |

Note bene: The derivative of the chain function is the product of the derivatives of the individual functions

- $\nabla z$ is a scalar, $y$ and $x$ are vectors
- The derivative of $\mathbf{z}$ w.r.t $y$ is going to be a row vector $\nabla_y(\mathbf{z})$
- The derivative of $\mathbf{z}$ w.r.t $y$ is going to be a row vector $\nabla_y(\mathbf{z})$
- The derivative of $y$ w.r.t $x$ is going to be a matrix (Jacobian)
- In the end we'll get a row vector

**Vector Calculus Note 2: Chain rule**

- For nested functions we have the following chain rule

$$\mathbf{z}(\mathbf{y}(\mathbf{x})) \implies \nabla_{\mathbf{x}} \mathbf{z}(\mathbf{y}(\mathbf{x})) = \nabla_{\mathbf{y}} \mathbf{z}(\mathbf{y}(\mathbf{x})) \cdot J_{\mathbf{x}}(\mathbf{y}(\mathbf{x}))$$

| | |
|---|---|
| Step 1 | $\mathbf{y} = \mathbf{y}(\mathbf{x})$ |
| Step 2 | $\mathbf{z} = \mathbf{z}(\mathbf{y})$ |
| Step 3 | $\mathbf{z} = \mathbf{z}(\mathbf{y}(\mathbf{x}))$ |

Note bene: The derivative of the chain function is the product of the derivatives of the individual functions

**Vector Calculus Note 2: Chain rule**

- For nested functions we have the following chain rule

$$\mathbf{z}(\mathbf{y}(\mathbf{x})) \implies \nabla_{\mathbf{x}} \mathbf{z}(\mathbf{y}(\mathbf{x})) = J_{\mathbf{y}}(\mathbf{z}) \cdot J_{\mathbf{x}}(\mathbf{y}(\mathbf{x}))$$

| | |
|---|---|
| Step 1 | $\mathbf{y} = \mathbf{y}(\mathbf{x})$ |
| Step 2 | $\mathbf{z} = \mathbf{z}(\mathbf{y})$ |
| Step 3 | $\mathbf{z} = \mathbf{z}(\mathbf{y}(\mathbf{x}))$ |

Note bene: The derivative of the chain function is the product of the derivatives of the individual functions

**More calculus: Special Case**

- Scalar functions of affine functions

$$z = \mathbf{W}\mathbf{y} + b$$

$$J = J(\mathbf{y})$$

$$J_{\mathbf{y}}(\mathbf{z}) = \mathbf{W}$$

$$J_b(\mathbf{z}) = 1$$

Jacobian w.r.t bias is 1

- Note: the derivative shapes are the transposes of the shapes of $\mathbf{W}$ and $b$.

$$D = J(\mathbf{y})$$

$$D = J(\mathbf{W}\mathbf{y} + b)$$

$$\nabla_{\mathbf{y}} D = \nabla_{\mathbf{y}} J(\mathbf{W}\mathbf{y} + b)$$

$$\nabla_{\mathbf{y}} D = \nabla_{\mathbf{y}} J(\mathbf{W}\mathbf{y})$$

Application

**Special Case: Application to a network**

- Scalar functions of Affine functions

$$\mathbf{z}^L = \mathbf{W}^L \mathbf{y}^{L-1} + b^L$$

$$\mathbf{y}^L = f(\mathbf{z}^L)$$

The Jacobian is only the activation function, applying the scalar one

$$\nabla_{\mathbf{z}^L} \mathbf{y}^L = \nabla_{\mathbf{z}^L} f(\mathbf{z}^L)$$

- $\mathbf{y}_L$ is influencing the divergence
- The derivative of the divergence wrt $\mathbf{y}_L$ is

$$\nabla_{\mathbf{y}^L} Div = \nabla_{\mathbf{y}^L} f(Div) \circ \mathbf{W}_L$$

**Special Case: Application to a network**

- Scalar functions of affine functions

$$\mathbf{z}^L = \mathbf{W}^L \mathbf{y}^{L-1} + b^L$$

$$\mathbf{y}^L = f(\mathbf{z}^L)$$

The Jacobian is only the activation function, applying the scalar one

$$\nabla_{\mathbf{z}^L} \mathbf{y}^L = \nabla_{\mathbf{z}^L} f(\mathbf{z}^L)$$

- $\nabla_{\mathbf{z}^L} \mathbf{y}^L$ is a diagonal matrix
- Jacobian $\mathbf{z}^L$ w.r.t $\mathbf{y}^{L-1}$ is just the transpose of $\mathbf{W}^L$
- Jacobian $\mathbf{z}^L$ w.r.t $b^L$ is just 1
- Jacobian $\mathbf{z}^L$ w.r.t $\mathbf{W}^L$ is just $\mathbf{y}^{L-1}$

## 4 Backpropagation

- After doing based pass and computing all the intermediate variables, then compute divergence which is a scalar
- The Derivative of Divergence wrt the output of the Network $\mathbf{Y}$ is calculated
- $\mathbf{Y}$ is a convex vector
- So $\nabla_{\mathbf{Y}} Div$ is going to be a row vector

**The backward pass**

Front computes the derivative of the divergence w.r.t $\mathbf{Y}$

$$\nabla_{\mathbf{Y}} Div = \nabla_{\mathbf{Y}} f(\mathbf{z}^N)$$

THE the gradient is the Jacobian of the activation

- Then the derivative of the Divergence wrt $z$ is computed $\nabla_{z^N} Div$
- Since $z = y - D$ and we already know $\nabla_{\mathbf{Y}} Div$, so we multiply the derivative of $\mathbf{Y}$ w.r.t to the right which is a gradient

$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \nabla_{z^N} y$$

**The backward pass**

The Divergence is a scalar function $Div(\mathbf{Y}, d_{\mathbf{Y}})$

$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot \nabla_{z^N} \mathbf{Y}$$

Chain rule

- $\nabla_{z^N} \mathbf{Y} = J(z_N)$
- Right multiply the result with the new term to give the new derivative

**The backward pass**

The Divergence is a scalar function $Div(\mathbf{Y}, d_{\mathbf{Y}})$

$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot J(z_N)$$

- $\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot \mathbf{W}_L$

**The backward pass**

$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot \mathbf{W}_L$$

**The backward pass**

$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot \mathbf{W}_L$$

**The backward pass**

$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot \mathbf{W}_L$$

**The backward pass**




$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot \mathbf{W}_L$$

**The backward pass**




$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot \mathbf{W}_L$$

**The backward pass**




$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot \mathbf{W}_L$$

**The backward pass**




$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot \mathbf{W}_L$$

**The backward pass**




$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot \mathbf{W}_L$$

**The backward pass**




$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot \mathbf{W}_L$$

**The backward pass**

$$\nabla_{z^N} Div = \nabla_{\mathbf{Y}} Div \cdot \mathbf{W}_L$$

**The Backward Pass**

- Let $\mathbf{y}_L = \mathbf{Y}$, $\mathbf{y}_0 = \mathbf{X}$
- Let $\mathbf{z}_L$ be the output of the network
- For layer $k = L$ backwards, $L-1$ to 1
  - Compute $\nabla_{\mathbf{z}_k} Div$
  - Compute $\nabla_{\mathbf{W}_k} Div$ and $\nabla_{\mathbf{b}_k} Div$ using the Forward Pass
    - Backward recursion step: $\nabla_{\mathbf{z}_k} Div = \nabla_{\mathbf{z}_k} \mathbf{y}_k \nabla_{\mathbf{y}_k} \mathbf{z}_{k+1} \nabla_{\mathbf{z}_{k+1}} Div$
    - Gradient computation: $\nabla_{\mathbf{W}_k} Div = \mathbf{y}_{k-1} \nabla_{\mathbf{z}_k} Div$
    - $\nabla_{\mathbf{b}_k} Div = \nabla_{\mathbf{z}_k} Div$

**For comparison: The Forward Pass**

- Set $\mathbf{y}_0 = \mathbf{x}$
- For layer $k = 1$ to $N$
  - Forward: $\mathbf{z}_k = \mathbf{W}_k \mathbf{y}_{k-1} + \mathbf{b}_k$
    $\mathbf{y}_k = f_k(\mathbf{z}_k)$
  - Output:
    $\mathbf{Y} = \mathbf{y}_N$

# Neural network training algorithm

- Initialize all weights and biases ($\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \ldots, \mathbf{W}_N, \mathbf{b}_N$)
- Do:
  - $Loss = 0$
  - For all $k$, initialize $\nabla_{\mathbf{W}_k} Loss = 0, \nabla_{\mathbf{b}_k} Loss = 0$
  - For all $t$ = 1:Number of Training instances
    - Forward pass: Compute
      - Output $\mathbf{Y}(\mathbf{X}_t)$
      - Loss $+= Div(\mathbf{Y}(\mathbf{X}_t), \mathbf{d}_t)$
    - Backward pass: For all $k$ compute:
      - $\nabla_{\mathbf{z}_N} \mathbf{y}_N = \nabla_{\mathbf{z}_N} \mathbf{Y}(\mathbf{X}_t)$
      - $\nabla_{\mathbf{y}_N} Div = \nabla_{\mathbf{Y}(\mathbf{X}_t)} Div(\mathbf{Y}(\mathbf{X}_t), \mathbf{d}_t)$
      - $\nabla_{\mathbf{z}_k} Div = \nabla_{\mathbf{z}_k} \mathbf{y}_k \nabla_{\mathbf{y}_k} \mathbf{z}_{k+1} \nabla_{\mathbf{z}_{k+1}} Div$, $\nabla_{\mathbf{W}_k} Div(\mathbf{X}_t, \mathbf{d}_t) = \mathbf{y}_{k-1} \nabla_{\mathbf{z}_k} Div$
      - $\nabla_{\mathbf{b}_k} Div = \nabla_{\mathbf{z}_k} Div$, $\nabla_{\mathbf{b}_k} Div(\mathbf{X}_t, \mathbf{d}_t) = \nabla_{\mathbf{z}_k} Div$
    - For all $k$, update:
      - $\mathbf{W}_k = \mathbf{W}_k - \eta \left[\nabla_{\mathbf{W}_k} Loss\right]^T$, $\mathbf{b}_k = \mathbf{b}_k - \eta \left[\nabla_{\mathbf{b}_k} Loss\right]^T$
- Until $Loss$ has converged



- We average the derivatives

# Story so far

- Neural networks must be trained to minimize the average divergence between the output of the network and the desired output over a set of training instances, with respect to network parameters.
- Minimization is performed using gradient descent
- Gradients (derivatives) of the divergence (for any individual instance) w.r.t. network parameters can be computed using backpropagation
  - Which requires a "forward" pass of inference followed by a "backward" pass of gradient computation
- The computed gradients can be incorporated into gradient descent